

데이터베이스

□ 데이터 관리

- 프로그램이 종료되면 변수에 저장된 정보는 사라짐
- HTTP는 요청 간에 상태를 기억하지 않는 무상태 프로토콜 (Stateless)
- 사용자 정보, 게시물, 상품 목록 등을 지속적으로 저장하려면 데이터베이스(Database) 필요

□ 다양한 DBMS 종류

- 관계형 데이터베이스 (RDBMS)
 - MySQL / MariaDB / Oracle / MSSQL / SQLite
- 비관계형 (NoSQL)
 - MongoDB 등

□ Node.js 데이터베이스 작업 흐름 개요

- 모듈(드라이버) 불러오기
- 데이터베이스 연결 객체 생성
- SQL 준비 및 실행
- 결과 처리
- 에러 및 종료 처리

□ mysql2 모듈 설치

npm install mysql2

□ 연결 객체 생성

```
const connection = await mysql.createConnection({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp'  
});
```

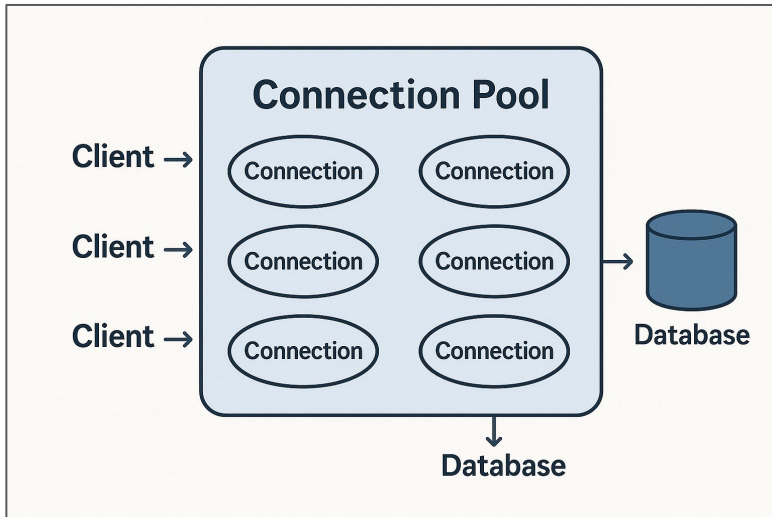
```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

□ 연결 풀(connection pool)

- 여러 개의 DB 연결을 미리 만들어 놓고 요청마다 빌려쓰는 방식
- 사용 후 연결을 닫지 않고 다시 풀에 반환

□ 연결 풀을 사용하지 않으면?

- 요청마다 새로운 연결 생성 → 느림, 리소스 낭비
- 연결이 많아질수록 서버 과부하 발생



□ 연결 풀을 사용하지 않는 방법 (직접 연결)

- 사용 시점마다 연결을 새로 생성하고, 명시적으로 종료해야 함

```
import mysql from 'mysql2/promise';
```

```
const connection = await mysql.createConnection({
  host: 'localhost', user: 'root', password: '1234',
  database: 'myapp'
});
const [rows] = await connection.execute('SELECT * FROM users');
await connection.end(); // 명시적으로 연결 종료
console.log(rows);
```

□ 연결 풀을 사용하는 방법 (추천 방식)

- 한 번 생성한 풀을 재사용하며, 매 요청마다 연결을 빌려 쿼리를 실행

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({
  host: 'localhost', user: 'root', password: '1234',
  database: 'myapp',
  waitForConnections: true, connectionLimit: 10,
});
const [rows] = await pool.execute('SELECT * FROM users');
await pool.end(); // 모든 연결을 종료하고 이벤트 루프에서 풀 제거
console.log(rows);
```

[실습]

연결 해제를 생략했을 경우 테스트

○ leak.js

```
import mysql from 'mysql2/promise';
```

```
const leakTest = async () => {  
  const connections = [];  
  for (let i = 0; i < 200; i++) {  
    try {  
      const conn = await mysql.createConnection({  
        host: 'localhost',  
        user: 'root',  
        password: '1234',  
        database: 'myapp'  
      });  
      connections.push(conn); // 종료하지 않고 배열에 저장만  
      console.log(`연결 ${i + 1}번 성공`);  
    } catch (e) {  
      console.error(`연결 ${i + 1}번 실패:`, e.message);  
  
      break;  
    }  
  }  
};
```

```
leakTest();
```

PROBLEMS	OUTPUT	TERMINAL	...
연결 143번	성공		
연결 144번	성공		
연결 145번	성공		
연결 146번	성공		
연결 147번	성공		
연결 148번	성공		
연결 149번	성공		
연결 150번	성공		
연결 151번	실패: Too many connections		

○ 데이터베이스 확인

-- DB에 접속할 수 있는 최대 클라이언트 수

```
SHOW VARIABLES LIKE 'max_connections';
```

Variable_name	Value
max_connections	151

-- 현재 DB에 연결되어 있는 클라이언트 수

```
SHOW STATUS WHERE Variable_name = 'Threads_connected';
```

Variable_name	Value
Threads_connected	2

-- 현재 DB에 연결되어 있는 클라이언트 정보

```
SHOW PROCESSLIST;
```

Id	User	Host	db	Command
407	root	172.17.0.1:60044	NULL	Query
408	root	172.17.0.1:60048	NULL	Sleep

□ await 사용

- `createConnection()`은 **Promise** 반환 → 반드시 `await` 필요

```
const connection = await mysql.createConnection({...});
```

→ 연결이 완료된 다음에만 쿼리를 실행할 수 있으므로 **await** 반드시 사용

- `createPool()`은 **Promise**가 아님 → `await` 필요 없음

```
const pool = mysql.createPool({...}); // 그냥 동기처럼 생성됨
```

→ 이건 단순히 **풀 객체**를 생성하는 함수이며, 내부적으로는 바로 연결하지 않음

메서드	반환값	설명
<code>createConnection()</code>	<code>Promise<Connection></code>	개별 연결을 직접 생성
<code>createPool()</code>	<code>Pool</code> 객체	연결 풀 구성 객체 반환
<code>pool.query()</code>	<code>Promise<[rows]></code>	실제 쿼리 실행 시 await

[실습]

테이블 생성

○ create-table.js

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

```
const createMember = async () => {  
  const sql = `CREATE TABLE IF NOT EXISTS users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50), age INT)`;  
  try {  
    await pool.execute(sql);  
    console.log("테이블 생성 완료");  
  } catch(e) {  
    console.error("테이블 생성 실패: ", e.message);  
  } finally {  
    await pool.end();  
    console.log("풀 종료");  
  }  
};  
createMember();
```

[실습]

데이터 입력

○ insert-user.js

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

```
const insertUser = async () => {  
  const sql = 'INSERT INTO users (name, age) VALUES (?, ?)';  
  try {  
    const [result] = await pool.execute(sql, ['nodejs', 20]);  
    console.log('사용자 삽입 완료:', result);  
  } catch (e) {  
    console.error('삽입 실패:', e.message);  
  } finally {  
    await pool.end();  
    console.log('풀 종료');  
  }  
};  
insertUser();
```

[실습]

데이터 조회

○ select-user.js

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

```
const getUsers = async () => {  
  try {  
    const [rows] = await pool.execute('SELECT * FROM users');  
    console.log(rows);  
  } catch (e) {  
    console.error('조회 실패:', e.message);  
  } finally {  
    await pool.end();  
    console.log('풀 종료');  
  }  
};  
  
getUsers();
```

[실습]

데이터 수정

○ update-user.js

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

```
const updateUser = async () => {  
  const sql = 'UPDATE users SET name = ? WHERE id = ?';  
  try {  
    const [result] = await pool.execute(sql, ['수정된이름', 1]);  
    console.log('수정 완료:', result);  
  } catch (e) {  
    console.error('수정 실패:', e.message);  
  } finally {  
    await pool.end();  
    console.log('풀 종료');  
  }  
};  
  
updateUser();
```


[실습]

데이터 삭제

○ delete-user.js

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '1234',  
  database: 'myapp',  
  waitForConnections: true,  
  connectionLimit: 10,  
});
```

```
const deleteUser = async () => {  
  const sql = 'DELETE FROM users WHERE id = ?';  
  try {  
    const [result] = await pool.execute(sql, [1]);  
    console.log('삭제 완료:', result);  
  } catch (e) {  
    console.error('삭제 실패:', e.message);  
  } finally {  
    await pool.end();  
    console.log('풀 종료');  
  }  
};  
  
deleteUser();
```

연습문제

1. member 데이터 제어하기

- 테이블 생성 ([create-member.js](#))
- 데이터 입력 ([insert-member.js](#))
- 데이터 조회 ([select-member.js](#))
- 데이터 수정 ([update-member.js](#))
- 데이터 삭제 ([delete-member.js](#))

- 테이블 구조

테이블명	컬럼명	자료형	설명
member	id	INT AUTO_INCREMENT PRIMARY KEY	회원 고유 ID
	name	VARCHAR(50)	회원 이름
	email	VARCHAR(100)	이메일 주소

- 기본 코드

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost', user: 'root', password: '1234',  
  database: 'myapp',  
  waitForConnections: true, connectionLimit: 10,  
});
```

```
const 함수명 = async () => {  
  const sql = 'SQL 작성';  
  try {  
    // 코드 작성  
  } catch (e) {  
    // 코드 작성  
  } finally {  
    // 코드 작성  
  }  
};  
함수명();
```

연습문제

2. article 데이터 제어하기

- 테이블 생성 ([create-article.js](#))
- 데이터 입력 ([insert-article.js](#))
- 데이터 조회 ([select-article.js](#))
- 데이터 수정 ([update-article.js](#))
- 데이터 삭제 ([delete-article.js](#))

- 테이블 구조

테이블명	컬럼명	자료형	설명
article	id	INT AUTO_INCREMENT PRIMARY KEY	게시글 고유 ID
	title	VARCHAR(100)	게시글 제목
	content	VARCHAR(1000)	게시글 내용
	member_id	INT	작성자 ID

- 기본 코드

```
import mysql from 'mysql2/promise';
```

```
const pool = mysql.createPool({  
  host: 'localhost', user: 'root', password: '1234',  
  database: 'myapp',  
  waitForConnections: true, connectionLimit: 10,  
});
```

```
const 함수명 = async () => {  
  const sql = 'SQL 작성';  
  try {  
    // 코드 작성  
  } catch (e) {  
    // 코드 작성  
  } finally {  
    // 코드 작성  
  }  
};  
함수명();
```